

Making MyFriendBen Safe for AI to Build On

Elvis Obondo, co-authored with Claude (Anthropic)

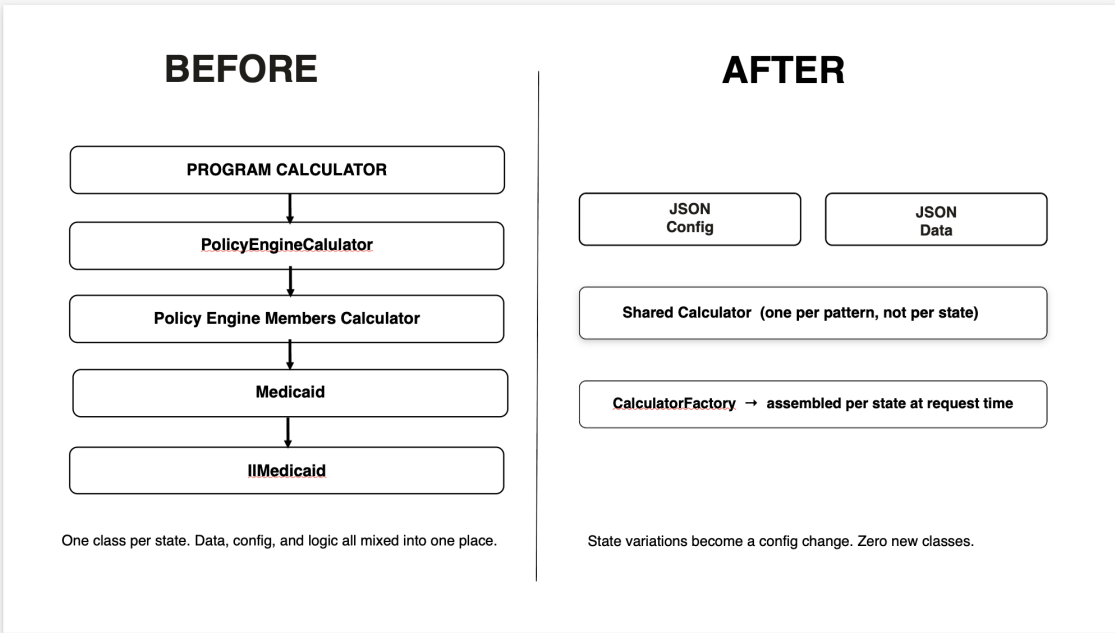
The problem, as raised

In October 2025, MyFriendBen engineer Caton Brewster wrote RFC 0003, an architecture proposal that sat with no real engagement for nine months. His diagnosis was simple: adding one benefits program to one state means writing a new calculator class, up to four levels deep in an inheritance chain, mixing data, configuration, and logic into a single file. His own worked example was Illinois Medicaid: understanding that one state's logic meant tracing through five files and four levels of inheritance. Testing meant mocking that entire chain plus live API calls.

This is exactly the kind of repetitive, well-scoped work AI could speed up. But handing a fast-moving agent a codebase where a mistake can hide five layers deep isn't a shortcut, it's a liability. Before any of that could happen safely, the architecture itself had to change.

Mapping the problem to a solution

The fix pulls apart the three things RFC 0003 found tangled together. Data and configuration become JSON, no code, no inheritance, just values. Logic stays in one shared calculator class per pattern (not per state). A factory assembles the right calculator for the right state at request time.



The RFC's own worked example was Medicaid. I deliberately didn't start there. Medicaid isn't one pattern, it's at least three different shapes across MyFriendBen's states, and building the new architecture against it first would have tested the architecture and three hard edge cases at the same time. If something broke, there'd be no way to know which one caused it. So the pilot started with SNAP and TANF, the two programs that are pure pass-through (PolicyEngine determines both eligibility and the dollar amount) in every state, zero exceptions. Prove the mechanism on the simple case, then climb the difficulty curve on purpose.

This also wasn't a blank-page rebuild. The RFC's own Alternatives Considered section had already weighed and rejected auto-discovering calculators by scanning directories, in favor of explicit, one-line-per-state registration, for real reasons around debugging clarity and phased rollout control. That reasoning held up, so the project built on it instead of re-litigating something already settled.

Technical challenges, and where the RFC's own questions get answered

RFC 0003 ends with two open questions the author didn't have time to resolve. Both get answered here with evidence instead of guesses.

Is the new client calling PolicyEngine's API twice? Grepping the real orchestration code for every type check it performs on a calculator turns up exactly two `isinstant` checks in the whole file, and neither one even inspects the calculator, they check dependency classes. The real system doesn't care what type a calculator is, it just calls the methods it needs. That meant the new calculator could plug into the same live PolicyEngine simulation the old calculator already created. Zero new API calls. An earlier version of the design gave the new calculator its own separate simulation call, and that was rejected specifically because it would have reintroduced the exact risk the RFC flagged.

How do state-level mixins fit into the new config system? A search across the entire codebase for mixin classes turns up exactly two, and both belong to fully custom programs that never touch PolicyEngine at all. Zero PolicyEngine-backed program anywhere uses a mixin. Rather than design a speculative mechanism for a case that doesn't exist, this is flagged honestly as an open gap and left undesigned. Building machinery for a problem you can't point to is exactly the kind of risk this project set out to avoid.

One more thing worth noting: the RFC's own code samples used plain Python classes for the data and config layers. This project moved that layer to JSON instead, for three reasons. JSON diffs cleanly in a pull request, a reviewer sees exactly which dollar amount changed, not a class body. It can be validated automatically before it reaches a calculator. And it means something other than a person reading Python can eventually read, and even generate, this configuration directly.

Adding a state to an existing program: Illinois SNAP

Illinois was deliberately left as the last of MyFriendBen's seven states to add to SNAP, specifically to demonstrate this live. Adding it required a one-line production registration, a small JSON config file, a five-line entry in the test harness, and a genuinely new fixture of Illinois-specific test scenarios (five scenarios across four Illinois cities, plus a categorical-eligibility stress case).

That last piece is the one place this project doesn't claim a free win. Illinois has a real, state-specific eligibility rule (a broad-based categorical eligibility waiver) that isn't just a config value, it's a fact about Illinois law that has to be researched and gotten right. Secondary sources disagreed with each other on the exact threshold (165% vs. 200% of the federal poverty line), so the real number was resolved directly against PolicyEngine's own source parameters instead of trusting either secondary source: 165% FPL, no net income test, no asset test, the same full waiver shape as Washington and North Carolina. The recorded test result confirmed it exactly, with no surprises. That's the honest bar for this kind of addition. The architecture didn't generate the answer, it made it fast to add, and the fact still had to be checked by hand.

Adding a state to a second program: Washington TANF

Illinois SNAP proves the architecture holds for a new state within a program it already knows. Washington TANF proves something different: that the same shared calculator generalizes to a second program entirely, without needing new logic.

Before adding any real TANF state, the project checked whether TANF even needed its own calculator class, rather than assuming SNAP's would work. Direct comparison of every state's existing TANF code confirmed none of them override the core calculation method, meaning the generic calculator built for SNAP already covers TANF with zero new Python. Texas was added first, chosen specifically because it had a real, verifiable old calculator to check the new one against, unlike Kansas, which has no existing TANF calculator in the codebase at all and so offers nothing to verify against yet.

Washington was originally supposed to be the second TANF state's Colorado, but Colorado's real production TANF calculator has a known accuracy issue (more on that below), so registering it would have produced an expected, already-explained mismatch rather than a genuine confirmation. Washington was checked directly and confirmed to be pure pass-through, same shape as Texas, and became the real second state instead.

Building it surfaced two things worth being direct about. First, a real fixture file for Washington's TANF program already existed in the codebase, recorded from actual past production data, and very nearly got overwritten by a freshly authored one before a routine check caught that it was already real. Second, the live-recorded result for Washington's TANF benefit didn't match the fixture's original hand-computed value (\$4,922 a year against a recorded \$6,672). Both the old, unmodified production calculator and the new one agree with each other exactly at the lower figure, which points to PolicyEngine's own underlying parameters having shifted since the fixture was first computed, not a bug in either calculator. It's left open rather than silently resolved, flagged for whoever eventually re-baselines these fixtures.

What this project actually found, and what it means at scale

The most consequential result of this project isn't in the architecture at all. While investigating why some states' existing TANF calculators bypass PolicyEngine's own income formulas, direct comparison against PolicyEngine's real source code found that Illinois's production calculator sums a household's entire income when determining TANF eligibility, where PolicyEngine's actual formula only counts the tax filer or their spouse. In practice, that means a working teenager's paycheck could be incorrectly counting against their family's real eligibility for cash assistance today, in already-shipped code, unrelated to anything this project built.

This was reported upstream as its own pull request: three new tests, zero production changes, demonstrating the discrepancy with real recorded numbers rather than just describing it. Building that demonstration caught two more mistakes before they became public. The first pass assumed Colorado's and North Carolina's calculators had the same bug; a suspicious result on North Carolina's test turned out to mean its actual divergence was something else entirely (workers' compensation income, not gift income), caught by reading the raw recorded data rather than trusting a number that happened to match. The second was a test that appeared to show the opposite of the real bug, traced back to a mistake in how the test itself built its household data, not to anything wrong in the underlying system.

That's the actual argument for what this project could mean at scale. Every real catch here (a passing test that had verified nothing, a fixture that shouldn't have been overwritten, a bug demonstration that needed correcting twice before it was right) happened because verification kept getting applied to the verification itself, not just to the first draft of an answer. That's the part of this that has to stay deliberate as more of the codebase gets touched by AI. The architecture change makes adding states fast. It's the verification discipline underneath it that makes fast safe.

Where this stands

Five pull requests are open against MyFriendBen's real repository, all awaiting maintainer review:

1. [#1637](#): JSON schema for the config and data layers
2. [#1638](#): the PolicyEngine client wrapping the existing simulation object
3. [#1639](#): the calculator factory and adapter
4. [#1640](#): the Illinois TANF income-counting discrepancy, reported on its own, unrelated to the architecture
5. [#1641](#): the full SNAP and TANF pilot across all seven states, including Illinois SNAP and Washington TANF

A comment linking this work was also posted directly on RFC 0003's own discussion thread, closing the loop on a nine-month-old proposal with real, working code instead of another round of discussion.